សៀវភៅណែនាំលម្អិតអំពីរបៀបដំណើរការគម្រោង Cloud Computing

ការដំឡើង និងដំណើរការប្រព័ន្ធ Cloud ស្វ័យប្រវត្តិតាមរយៈ Docker Swarm, Terraform, Nginx និង Jenkins

អ្នករៀបចំគម្រោង៖ ចំរើន សារ៉ាន់ (Chamroeun Saran) **ថ្នាក់សិក្សា៖** Master of Science in Cloud Computing

តំណភ្ជាប់ Git៖ github.com/Chamroeun12/Assignment **តំបន់ AWS៖** Sydney (ap-southeast-2)

១. សេចក្តីផ្តើមអំពីស្ថាបត្យកម្មគម្រោង (System Architecture)

គម្រោងនេះត្រូវបានបង្កើតឡើងសម្រាប់កិច្ចការបញ្ចប់ថ្នាក់អនុបណ្ឌិត Cloud Computing ដោយរួមបញ្ចូលនូវបច្ចេកវិទ្យាសំខាន់ៗចំនួន ៥៖

- **Infrastructure as Code (Terraform)៖** បង្កើតប្រព័ន្ធបណ្តាញសុវត្ថិភាព VPC (10.0.0.0/16), Security Groups, និងម៉ាស៊ីនមេ AWS EC2 ចំនួន ៣ (១ Manager + ២ Workers) ដោយស្វ័យប្រវត្តិ។
- **Container Orchestration (Docker Swarm)៖** គ្រប់គ្រងកុងតឺន័រនៅលើម៉ាស៊ីនមេចំនួន ៣ ជាមួយការផ្លាស់ប្តូរកំណែថ្មីដោយគ្មានការដាច់សេវា (Zero-Downtime Rolling Update)។
- **Nginx Reverse Proxy & Load Balancing៖** ចែករំលែកបន្ទុកចរាចរណ៍ (Round-Robin Traffic) លើ Port 80 ទៅកាន់ Application Replicas។
- **CI/CD Pipeline (Jenkins)៖** ស្វ័យប្រវត្តិកម្មនៃការត្រួតពិនិត្យកូដ (Flake8), តេស្ត (Pytest), ស្កេនមេរោគ (Trivy), និងផ្ញើសារដំណឹងទៅ Telegram Bot។

២. តម្រូវការប្រព័ន្ធមុនពេលចាប់ផ្តើម (Prerequisites)

កម្មវិធី / ឧបករណ៍	សម្រាប់រត់លើ Laptop ផ្ទាល់ខ្លួន	សម្រាប់រត់លើ AWS EC2 Cloud	ការណែនាំ
Git	ចាំបាច់ត្រូវមាន	ចាំបាច់ត្រូវមាន	កំណែ 2.30+ សម្រាប់ទាញយកកូដ
Docker Desktop	ចាំបាច់ត្រូវមាន	មិនចាំបាច់មានទេ	កំណែ 24.0+ (រួមមាន Docker Compose)
AWS CLI	មិនបាច់មាន	ចាំបាច់ត្រូវមាន	កំណត់សិទ្ធិ Access Key & Secret Key
Terraform	មិនបាច់មាន	ចាំបាច់ត្រូវមាន	កំណែ 1.5.0+ សម្រាប់បង្កើត Server
SSH Key (.pem)	មិនបាច់មាន	ចាំបាច់ត្រូវមាន	ឯកសារ `chamroeunsaran.pem` សម្រាប់ Login

៣. វិធីសាស្ត្រទី ១៖ ការរត់តេស្តលើ Laptop ផ្ទាល់ខ្លួន (Docker Compose)

ប្រសិនបើអ្នកមិនចង់ចំណាយលុយលើ AWS ឬគ្មានគណនី AWS អ្នកអាចរត់សាកល្បង Full Stack (Flask + Nginx) លើកុំព្យូទ័ររបស់អ្នកក្នុងរយៈពេលត្រឹមតែ ៣០ វិនាទី៖

ជំហាន ៣.១៖ ទាញយកកូដពី GitHub (Clone Repo)

```
git clone https://github.com/Chamroeun12/Assignment.git
cd Assignment
```

ជំហាន ៣.២៖ បញ្ជាឱ្យដំណើរការកុងតឺន័រទាំងអស់

```
docker compose up --build -d
```

ជំហាន ៣.៣៖ ពិនិត្យមើលស្ថានភាពកុងតឺន័រ

```
docker compose ps
```

អ្នកនឹងឃើញកុងតឺន័រទាំងពីរដំណើរការធម្មតា៖ `cicd-assignment-web` (Flask Backend) និង `cicd-assignment-nginx` (Nginx Reverse Proxy)។

ជំហាន ៣.៤៖ តេស្តមើលលទ្ធផលនៅលើ Browser

បើកកម្មវិធី Browser (Chrome/Edge) ហើយវាយអាសយដ្ឋាន៖ `http://localhost/`

```
curl -i http://localhost/            # បង្ហាញ Dashboard លើ Web
curl -s http://localhost/health       # បង្ហាញស្ថានភាពសុខភាពប្រព័ន្ធ
curl -s http://localhost/api/info     # បង្ហាញព័ត៌មានលម្អិត និង Hostname
curl -s http://localhost/nginx-health # បញ្ជាក់ថា Nginx ដំណើរការល្អ
```

ជំហាន ៣.៥៖ បិទប្រព័ន្ធពេលឈប់ប្រើ

```
docker compose down
```

៤. វិធីសាស្ត្រទី ២៖ ការដំឡើង និងដំណើរការលើ AWS EC2 Cloud (Terraform & Docker Swarm)

នេះជាការដំឡើងជាក់ស្តែងលើម៉ាស៊ីនមេ AWS EC2 ចំនួន ៣ ដូចនៅក្នុងការធ្វើបទបង្ហាញថ្នាក់រៀន៖

ជំហាន ៤.១៖ កំណត់សិទ្ធិគណនី AWS CLI

```
aws configure
# វាយបញ្ចូល AWS Access Key ID របស់អ្នក
# វាយបញ្ចូល AWS Secret Access Key របស់អ្នក
# Default region: ap-southeast-2 (Sydney)
# Default output format: json
```

ជំហាន ៤.២៖ បង្កើតហេដ្ឋារចនាសម្ព័ន្ធដោយស្វ័យប្រវត្តិតាមរយៈ Terraform

```
cd terraform
terraform init
terraform plan
terraform apply -auto-approve
```

នៅពេលដំណើរការចប់ Terraform នឹងបង្ហាញ IP ម៉ាស៊ីនមេទាំង ៣ ដោយស្វ័យប្រវត្តិ៖

```
Outputs:
manager_public_ip = "16.178.206.23"
manager_private_ip = "172.31.34.160"
worker_public_ips = ["3.107.96.140", "13.211.213.228"]
worker_private_ips = ["172.31.36.22", "172.31.47.183"]
application_url = "http://16.178.206.23"
```

```
cd ..
```

ជំហាន ៤.៣៖ ចាប់ផ្តើម Docker Swarm និងភ្ជាប់ Worker Nodes

១. ចូលទៅកាន់ Manager Node តាមរយៈ SSH៖

```
ssh -i chamroeunsaran.pem ubuntu@16.178.206.23
```

២. ចាប់ផ្តើម Docker Swarm នៅលើ Manager៖

```
sudo docker swarm init --advertise-addr 172.31.34.160
sudo hostnamectl set-hostname swarm-manager
sudo systemctl restart docker
```

(ប្រព័ន្ធនឹងផ្តល់ Token មួយខ្សែសម្រាប់ឱ្យ Worker យកទៅ Join ដូចជា៖ `docker swarm join --token SWMTKN-1-... 172.31.34.160:2377`)

៣. ភ្ជាប់ Worker 1 ចូលទៅក្នុង Swarm៖

```
ssh -i chamroeunsaran.pem ubuntu@3.107.96.140
sudo docker swarm join --token <TOKEN> 172.31.34.160:2377
sudo hostnamectl set-hostname swarm-worker1
sudo systemctl restart docker
exit
```

៤. ភ្ជាប់ Worker 2 ចូលទៅក្នុង Swarm៖

```
ssh -i chamroeunsaran.pem ubuntu@13.211.213.228
sudo docker swarm join --token <TOKEN> 172.31.34.160:2377
sudo hostnamectl set-hostname swarm-worker2
```

```
sudo systemctl restart docker
exit
```

៥. ផ្ទៀងផ្ទាត់បញ្ជី Nodes នៅលើ Manager៖

```
ssh -i chamroeunsaran.pem ubuntu@16.178.206.23
sudo docker node ls
# អ្នកនឹងឃើញ 1 Manager (Leader) + 2 Workers ស្ថិតក្នុងស្ថានភាព Ready និង Active
```

ជំហាន ៤.៤៖ បញ្ជូន និងដាក់ឱ្យដំណើរការ Application Stack

ពីលើកុំព្យូទ័ររបស់អ្នក ត្រូវខ្ចប់កូដ ហើយផ្ញើទៅកាន់ម៉ាស៊ីនមេទាំង ៣៖

```
# ១. ខ្ចប់កូដគម្រោង
tar -czf app_bundle.tar.gz Dockerfile app.py requirements.txt templates docker-stack.yml nginx

# ២. បញ្ជូនកូដទៅកាន់ម៉ាស៊ីនទាំង ៣ តាម SCP
scp -i chamroeunsaran.pem app_bundle.tar.gz ubuntu@16.178.206.23:/home/ubuntu/
scp -i chamroeunsaran.pem app_bundle.tar.gz ubuntu@3.107.96.140:/home/ubuntu/
scp -i chamroeunsaran.pem app_bundle.tar.gz ubuntu@13.211.213.228:/home/ubuntu/

# ៣. Build រូបភាពកុងតឺន័រលើ Worker 1 និង Worker 2
ssh -i chamroeunsaran.pem ubuntu@3.107.96.140 "tar -xzf app_bundle.tar.gz && sudo docker build -t 3993118152
ssh -i chamroeunsaran.pem ubuntu@13.211.213.228 "tar -xzf app_bundle.tar.gz && sudo docker build -t 39931181

# ៤. Deploy Stack នៅលើ Manager Node
ssh -i chamroeunsaran.pem ubuntu@16.178.206.23
tar -xzf app_bundle.tar.gz
sudo docker build -t 399311815210.dkr.ecr.ap-southeast-2.amazonaws.com/cicd-assignment:latest .
sudo docker stack deploy -c docker-stack.yml cicd-assignment
```

ជំហាន ៤.៥៖ ពិនិត្យមើលលទ្ធផលនៅលើ Browser និង Terminal

```
sudo docker service ls
sudo docker stack ps cicd-assignment
```

បើកមើលលើ Browser៖ `http://16.178.206.23/`

```
curl -s http://16.178.206.23/health
curl -s http://16.178.206.23/api/info        # Hostname នឹងឆ្លាស់គ្នាបញ្ជាក់ពី Load Balancing
curl -s http://16.178.206.23/nginx-health
```

៥. វិធីសាស្ត្រទី ៣៖ ការប្រើប្រាស់ Jenkins CI/CD ស្វ័យប្រវត្តិកម្ម

ប្រសិនបើអ្នកចង់បើកមើល Jenkins ដែលមាន Build History ទាំងអស់ (Build #7 ដល់ #21) នៅលើម៉ាស៊ីនថ្មី៖

```
# ១. បង្កើត Container Jenkins ថ្មី
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:l

# ២. ផ្ទេរទិន្នន័យ Backup ចូលទៅក្នុង Container
docker cp jenkins_backup.tar.gz jenkins:/tmp/jenkins_backup.tar.gz
docker exec -u 0 jenkins tar xzf /tmp/jenkins_backup.tar.gz -C /var/jenkins_home
docker exec -u 0 jenkins chown -R jenkins:jenkins /var/jenkins_home

# ៣. ដោះសោរ Security ដើម្បីកុំឱ្យទាមទារ Password ពេល Demo
docker exec -u 0 jenkins sed -i 's/<useSecurity>true<\/useSecurity>/<useSecurity>false<\/useSecurity>/' /var
docker restart jenkins

# ៤. បើក Browser មើល Jenkins Job ភ្លាមៗ៖
http://localhost:8080/job/Assignment-CI-CD/
```

៦. ការរត់តេស្តគុណភាពកូដ (Testing & Linting)

```
# បង្កើត Virtual Environment សម្រាប់ Python
python -m venv .venv
source .venv/bin/activate  # លើ Windows ប្រើ៖ .venv\Scripts\Activate.ps1

# ដំឡើងបណ្ណាល័យជំនួយ
pip install -r requirements.txt
pip install -r requirements-dev.txt

# រត់ត្រួតពិនិត្យទម្រង់កូដតាមស្តង់ដារ PEP 8
flake8 --max-line-length=120 --exclude=venv,.venv,build_submission_report.py,generate_evidence_screens.py ap

# រត់តេស្តស្វ័យប្រវត្តិកម្ម Pytest ជាមួយ Coverage
pytest -v --cov=app --cov-report=term-missing tests/
```

លទ្ធផលទទួលបាន៖ ជាប់ 100% (5 passed) ជាមួយកម្រិត Code Coverage 89%។

៧. ការគ្រប់គ្រងការចំណាយ និងការបិទម៉ាស៊ីនមេ (Cost Management)

> **ការប្រុងប្រយ័ត្ន៖** AWS គិតលុយតាមម៉ោងដំណើរការរបស់ EC2 និងទំហំផ្ទុក Hard Disk (EBS)។ សូមបិទ ឬលុបម៉ាស៊ីនមេពេលដែលធ្វើបទបង្ហាញចប់!

ជម្រើសទី ១៖ ផ្អាកដំណើរការបណ្តោះអាសន្ន (Stop Instances - មិនបាត់បង់ទិន្នន័យ)

```
aws ec2 stop-instances --instance-ids <manager_id> <worker1_id> <worker2_id> --region ap-southeast-2
```

ជម្រើសទី ២៖ លុបធនធានទាំងអស់ចោលជាស្ថាពរ (Terraform Destroy)

```
cd terraform
terraform destroy -auto-approve
```

៨. សំណួរ និងចម្លើយបច្ចេកទេស (FAQ)

- **សួរ៖ ហេតុអ្វីបានជា Public IP ផ្លាស់ប្តូរពេល Restart ម៉ាស៊ីន EC2?**
  ឆ្លើយ៖ តាមលំនាំដើម AWS ប្រើប្រាស់ Dynamic IP ពី Public Pool។ នៅពេល Stop ម៉ាស៊ីន AWS ដក IP យកទៅឱ្យអ្នកផ្សេងប្រើវិញ។ នៅពេល Start ឡើងវិញ ទើបផ្តល់ IP ថ្មី។ ប៉ុន្តែ Private IP នៅក្នុងបណ្តាញខាងក្នុងមិនផ្លាស់ប្តូរឡើយ ដូច្នេះ Docker Swarm ដំណើរការធម្មតា។

- **សួរ៖ តើត្រូវធ្វើដូចម្តេចដើម្បីតេស្តមើល Zero-Downtime Rolling Update?**
  ឆ្លើយ៖ ដំណើរការបញ្ជា៖ `sudo docker service update --image <IMAGE>:v2 --update-order start-first cicd-assignment_web` ។ Swarm នឹងបង្កើតកុងតឺន័រថ្មីឱ្យដើរស្រួលបួលសិន ទើបបិទកុងតឺន័រចាស់។
- **សួរ៖ តើត្រូវមើល Logs របស់ Flask App លើ Swarm យ៉ាងដូចម្តេច?**
  ឆ្លើយ៖ វាយពាក្យបញ្ជា៖ `sudo docker service logs -f cicd-assignment_web` ។